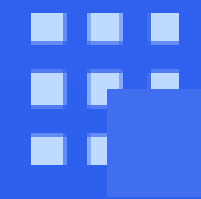


02

控制流图



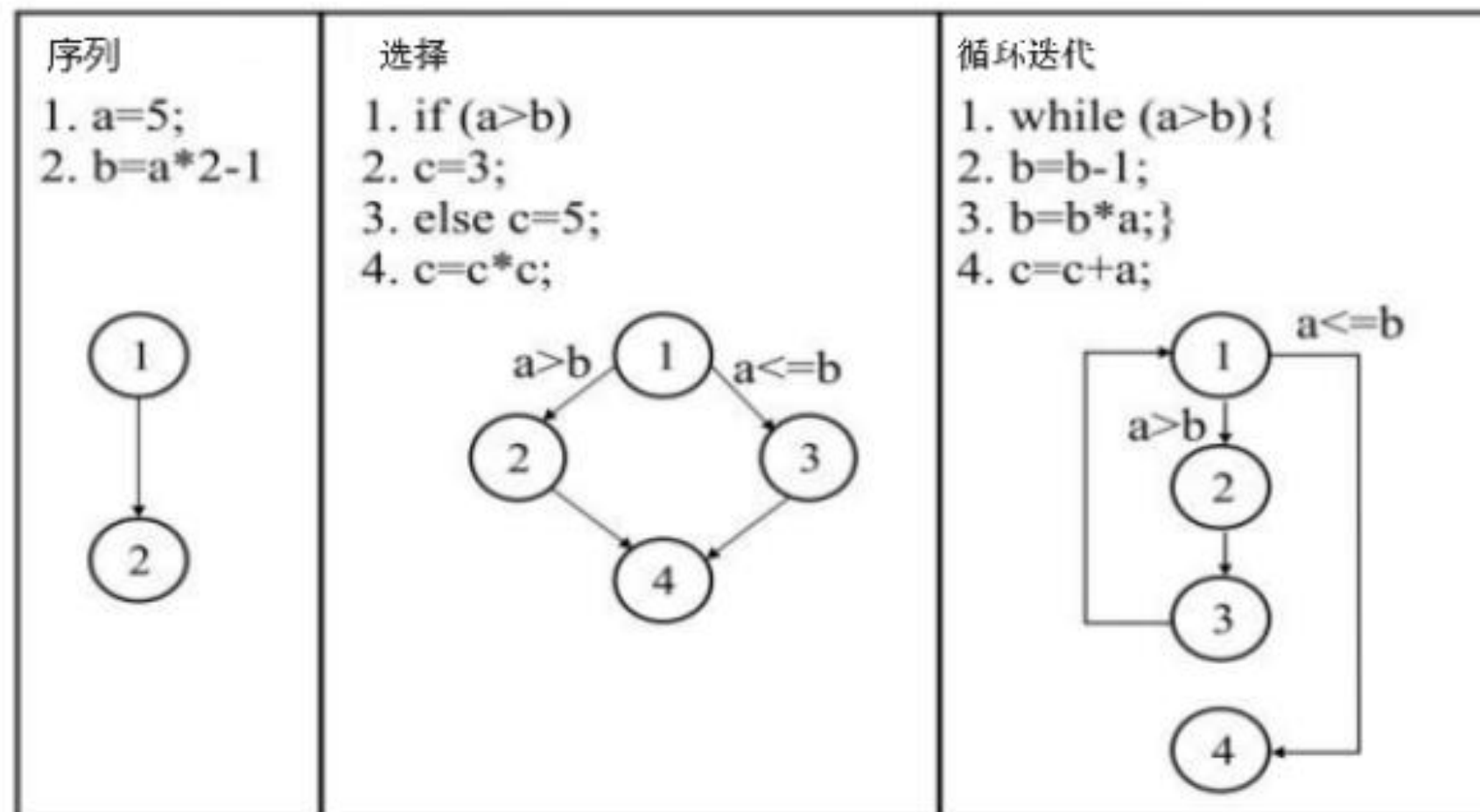
控制流图概述

源代码常常将可执行语句和分支映射成控制流图。控制流图是程序或应用程序执行期间控制流或计算行为的抽象图形化表示。控制流图可以准确地表示程序内部的流，因此主要用于支持静态分析和编译器应用。

定义 5.3: 控制流图 (Control Flow Graph, CFG)

程序控制流图 $CFG = (V, E, v\downarrow, v\uparrow)$ 对于程序中的每个语句 a 有一个顶点 $v_a \in V$ 。首先从 $v\downarrow$ 开始，如果语句 a' 在语句 a 之后立即执行，添加一条边 $(v_a, v_{a'})$ ，为每个与 a 关联的顶点 $v_{a'}$ 添加边之后控制流会因为返回语句或终止函数的右括号而离开函数，最终引向终结顶点 $v\uparrow$ 。

控制流图基本结构



控制流图能够展示程序执行期间可以遍历的结构和元素。控制流图是一个有向图，其中顶点对应于语句或基本块。大多数程序都是用这三种基本结构：顺序、选择和循环迭代。

图分析测试 ■ 控制流图

以max 函数为例。首先, 生成 CFG 的顶点。将使用每个程序语句对应一个顶点的简单近似。 用顶点号注释程序 (如左图) 。接下来按照程序语义, 通过互连具有顺序依赖关系 (如右图) 来连接顶点绘制控制流图。

```
1 int max(int a, int b) \\ 顶点1
2 if (a>b) \\ 顶点2
3     r=a; \\ 顶点3
4 else
5     r=b; \\ 顶点4
6 return r; \\ 顶点5
```

```
1 int max(int a, int b) \\ 1->2
2 if (a>b) \\ 2->3和2->4
3     r=a; \\ 3->5
4 else
5     r=b; \\ 4->5
6 return r;
```

控制流图生成基本步骤

1. 语法分析：构建语法树，以确定程序的语法结构，识别出各种语句、表达式等语法成分。
2. 语义分析：对语法树进行语义检查，确保程序符合语义规则，进行符号表的建立和维护，记录程序中各种符号的相关信息。
3. 中间代码生成：将经过语义分析的语法树转换为中间代码形式。
4. 控制流图构建：以中间代码为基础，构建控制流图。
5. 控制流优化：对生成的控制流图进行优化，合并基本块、消除无用的跳转等，以提高程序的执行效率。

练习：尝试手动生成Triangle程序的控制流图

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b == c:  
        print("等边三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

```
def next_day(year, month, day):
    tomorrow_month, tomorrow_day, tomorrow_year = month, day, year
    if month in [1, 3, 5, 7, 8, 10]:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month in [4, 6, 9, 11]:
        if day < 30:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month == 12:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = 1
            tomorrow_year = year + 1
```

练习：尝试手动生成NextDay程序的控制流图

```
elif month == 2:
    if day < 28:
        tomorrow_day = day + 1
    elif day == 28:
        if leap_year(year) == 1:
            tomorrow_day = 29
        else:
            tomorrow_day = 1
            tomorrow_month = 3
    elif day == 29:
        tomorrow_day = 1
        tomorrow_month = 3
return (tomorrow_year, tomorrow_month, tomorrow_day)
```

练习：尝试手动生成MeanVar程序的控制流图

```
public class MeanVar {  
    public static void MeanVar(double[] X) {  
        double var, mean, sum, varsum;  
        sum = 0;  
        int L = X.length;  
        for (int i = 0; i < L; i++) {  
            sum += X[i];  
        }  
        mean = sum / L;  
        varsum = 0;  
        for (int i = 0; i < L; i++) {  
            varsum += ((X[i] - mean) * (X[i] - mean));  
        }  
        var = varsum / (L - 1);  
        System.out.printf("Mean: %f\n", mean);  
        System.out.printf("Variance: %f\n", var);  
    }  
}
```


控制流图与程序优化

冗余代码消除：控制流分析能够识别出程序中那些不会被执行的代码段，也就是死代码。通过确定程序的执行路径，找出那些无论在何种输入情况下都不会被访问到的代码，将其删除可以减小程序的规模，提高程序的执行效率。

原始代码

```
x = 1
if x > 2:
    y = 3
    print(y)
```

优化后代码

```
x = 1
```

控制流图与程序优化

代码外提：控制流分析可以确定循环中哪些代码是不变的。通过将这些不变代码从循环内部移动到循环外部（代码外提），可以避免在每次循环迭代时重复计算，从而提高程序的性能。

原始代码

```
for i in range(10):  
    a = 2 + 3 # 每次循环都重复计算  
    b = a * i
```

优化后代码

```
a = 2 + 3  
for i in range(10):  
    b = a * i
```

回顾与讨论

- 控制流图定义
- 控制流图结构
- 控制流图生成



- 控制流图对软件测试的作用
- 控制流图对程序优化的作用



思考：神经网络的控制流图